PROYECTO INTERMODULAR 2

SPRINT 4 – INFORME DE GESTION DE SCRUM



Miriam Mendieta Racedo.-
2DAMS
03/11/2026

SPRINT 4 – INFORME DE GESTIÓN DE SCRUM

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 03 de noviembre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---------- **4**
EVOLUCIÓN DEL TRABAJO Y ESTADO DE HISTORIAS DE USUARIO (SPRINT 4) ---------- **5**
A. Historias de Usuario Completadas ---------- 7
B. Dificultades / Impedimentos Técnicos Solventados ---------- 11
Conclusiones del Sprint 4 ---------- **13**
BIBLIOGRAFÍA ---------- **15**

INTRODUCCIÓN

El testing de software es una disciplina fundamental dentro del ciclo de vida del desarrollo software, cuyo objetivo es verificar y validar que un sistema cumple los requisitos funcionales, no funcionales y de calidad establecidos. En el contexto de metodologías ágiles como Scrum, las pruebas no se consideran una fase aislada, sino una actividad continua que acompaña al desarrollo en cada Sprint.

Con este sprint, la aplicación queda lista para ser demostrada, desplegada y ampliada.

Los objetivos principales de sprint son:

- Garantizar la calidad del sistema: Implementar pruebas automáticas que validen el comportamiento crítico de la aplicación y prevengan regresiones.
- Estandarizar el despliegue: Empaquetar la aplicación en una imagen Docker funcional para permitir su ejecución en cualquier entorno sin dependencias locales.

❖ Pruebas automatizadas del sistema (Testing)

El propósito es asegurar que las funcionalidades principales del sistema funcionan correctamente y que no se producen regresiones al añadir nuevas características. Las pruebas automatizadas permiten:

- Validar la lógica de negocio.
- Comprobar que los controladores responden correctamente.
- Asegurar que la auditoría, seguridad y gestión de activos funcionan como se espera.
- Detectar errores antes del despliegue.
- Documentar el comportamiento esperado del sistema.

❖ Dockerización de la aplicación

El objetivo es generar una imagen Docker que encapsule la aplicación con todas sus dependencias, permitiendo ejecutarla en cualquier entorno sin configuraciones adicionales.

Evolución del Trabajo y Estado de Historias de Usuario (Sprint 4)

Los tests deben desarrollarse en paralelo al código porque permiten detectar errores en el momento en que se introducen, garantizan la coherencia del modelo, reducen la deuda técnica y evitan refactorizaciones costosas. Mi experiencia demuestra que dejar los tests para el final provoca una acumulación de fallos encadenados, dificulta el diagnóstico y aumenta el coste de corrección. La construcción de software de calidad exige que el desarrollo y las pruebas avancen en paralelo, no como fases separadas:

1. Los tests detectan errores en el momento en que se introducen → Cuando se programa sin tests, los errores se acumulan silenciosamente.
   Al escribir los tests a la vez que el código:
   - cada módulo se valida en cuanto se implementa,
   - los fallos se detectan en su contexto original,
   - y la corrección es inmediata y barata.

   En mi caso, al dejar los tests para el final, aparecieron errores encadenados: primero fecha_alta, luego categoria, luego estado, luego id_usuario. Si los tests hubieran existido desde el principio, cada error habría aparecido en el momento exacto en que se introdujo, no todos juntos al final.

2. Evitan deuda técnica y refactorizaciones dolorosas → Cuando el código crece sin pruebas, se vuelve frágil: cualquier cambio rompe algo.
   Los tests actúan como una red de seguridad que permite:
   - refactorizar sin miedo,
   - reorganizar entidades,
   - modificar repositorios,
   - añadir campos obligatorios,
   - y validar reglas de negocio.

   Al no tener tests, cada cambio en la entidad ActivoDigital generó fallos ocultos que solo aparecieron al final, obligando a revisar toda la estructura.

3. Garantizan que el diseño del modelo es coherente

Los tests obligan a pensar en:

- qué campos son obligatorios,
- qué valores deben tener los enums,
- qué relaciones deben existir,
- qué restricciones de integridad son necesarias.

En mi caso, solo al escribir los tests descubrí que:

- categoria era obligatoria,
- estado era obligatorio,
- id_usuario era obligatorio,
- y que el repositorio no funcionaba sin un contexto JPA completo.

Si los tests se hubieran escrito desde el principio, el diseño del modelo habría sido más claro y consistente.

4. Reducen el coste total del proyecto → Corregir un error cuando se introduce cuesta minutos.

Corregirlo semanas después cuesta horas o días, porque:

- hay que reconstruir el contexto,
- revisar código antiguo,
- entender dependencias,
- y corregir efectos secundarios.

Mi experiencia lo demuestra: los errores aparecieron en cascada y obligaron a revisar el modelo, el repositorio, el test y la configuración del proyecto.

5. Permiten validar que el sistema funciona antes de integrarlo

Los tests de repositorio, servicio y controlador garantizan que cada capa funciona por separado. Si se dejan para el final:

- no se sabe si el fallo está en la entidad,

- en el repositorio,
- en el servicio,
- en el controlador,
- o en la configuración.

En mi caso, el primer fallo parecía de JPA, luego de contexto, luego de integridad, luego de enums… Los tests habrían aislado cada problema desde el principio.

A. Historias de Usuario Completadas

HU – Implementación Completa de Pruebas del Sistema

Descripción

Disponer de pruebas automáticas que validen el comportamiento del sistema, para asegurar que las funcionalidades críticas funcionan correctamente y evitar regresiones antes del empaquetado en Docker.

Tareas técnicas

- Crear pruebas unitarias para servicios (activos, usuarios, auditoría).
- Crear pruebas de integración con MockMvc para controladores.
- Implementar pruebas de seguridad con roles (ADMINISTRACIÓN, AUDITORÍA, USUARIO).
- Crear pruebas de repositorio con H2 (@DataJpaTest).
- Validar auditoría y comportamiento de logs.

Criterios de aceptación

1. Pruebas funcionales:
   - Las rutas críticas tienen pruebas automáticas.
   - Las pruebas de seguridad validan accesos permitidos y denegados.

- Las pruebas de auditoría validan registros correctos y activos eliminados.
- No existen errores 500 durante la ejecución de pruebas.

2. Comportamiento consistente en todo el sistema
   - Las pruebas garantizan que el sistema responde igual en todos los entornos.
   - Las pruebas detectan regresiones antes del empaquetado.
   - La auditoría funciona correctamente con IDActivo = NULL.

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in com.miriammendieta.zeerty_digital
_app.ZeertyDigitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.5.1:jar (default-jar) @ zeerty_digital_app ---
[INFO] Building jar: D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app\target\zeerty_dig
ital_app-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:4.1.2:repackage (repackage) @ zeerty_digital_app ---
[INFO] Replacing main artifact D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app\target\
zeerty_digital_app-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_d
igital_app\target\zeerty_digital_app-0.0.1-SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  28.709 s
```

HU – Crear imagen Docker y empaquetar la aplicación

Descripción

Como administrador del sistema, quiero disponer de una imagen Docker funcional del proyecto, para poder desplegar la aplicación en cualquier entorno sin necesidad de configuraciones adicionales.

Tareas técnicas

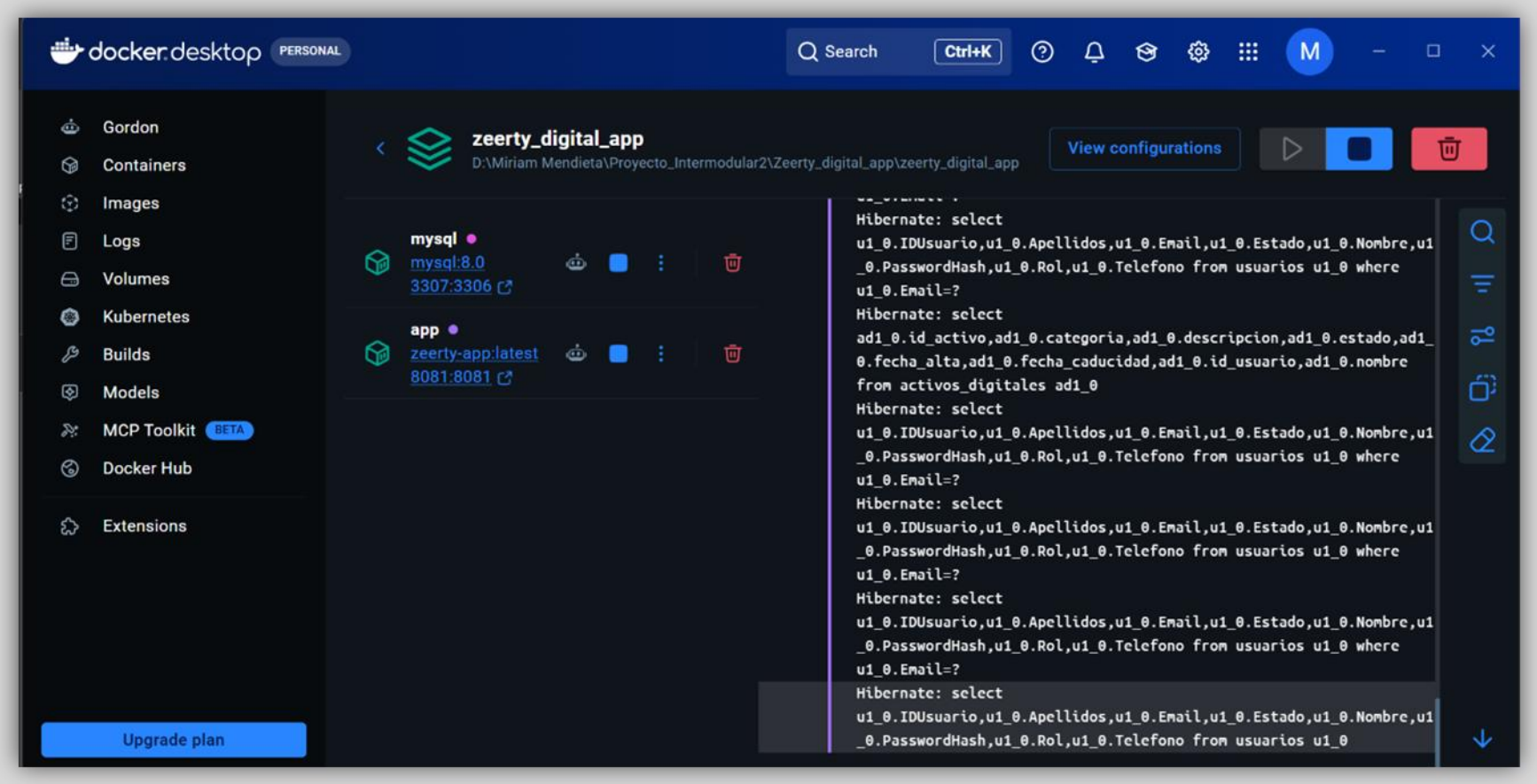
- Crear Dockerfile optimizado.
- Configurar application.properties para entorno Docker.
- Generar JAR con Maven/Gradle.
- Construir imagen Docker (docker build).
- Validar ejecución en contenedor (docker run).
- Documentar instrucciones de despliegue.
- Verificar logs y puertos expuestos.

Criterios de aceptación

1. Pruebas funcionales
   - La aplicación arranca correctamente dentro del contenedor.
   - El contenedor expone el puerto configurado.
   - La auditoría, seguridad y CRUD funcionan dentro de Docker.
   - No existen errores de dependencias ni rutas.

2. Comportamiento consistente en todo el sistema
   - La aplicación se comporta igual dentro y fuera de Docker.
   - El contenedor es reproducible en cualquier máquina.
   - La imagen es estable y lista para despliegue.

PROYECTO INTERMODULAR 2



B. Dificultades / Impedimentos Técnicos Solventados

Tipo de Impedimento	Descripción del Problema	Impacto en el Sprint	Estrategia de Solución	Medidas Preventivas / Lecciones Aprendidas
Pruebas funcionales (HU)	Las Historias de Usuario no tenían criterios de aceptación claros, lo que provocaba pruebas ambiguas y resultados inconsistentes.	Retrasos en validación y necesidad de rehacer pruebas.	Redefinir cada HU con criterios de aceptación medibles y reproducibles. Crear casos de prueba simples y repetibles.	Documentar criterios de aceptación desde el inicio. Validar cada HU con pruebas manuales antes de cerrar la tarea.
Pruebas de regresión	Cambios en vistas Thymeleaf rompían funcionalidades ya implementadas (login, navegación, atributos del modelo).	Reapertura de tareas ya completadas.	Crear un checklist de regresión: login, navegación, CRUD de activos, movimientos. Probar cada módulo tras cada cambio.	Mantener pruebas de regresión obligatorias antes de cada commit importante.
Pruebas de seguridad	Spring Security permitía iniciar sesión a usuarios desactivados.	Riesgo de seguridad y comportamiento incorrecto.	Añadir validación manual del estado del usuario en el servicio de autenticación.	Revisar configuraciones por defecto del framework. Documentar reglas de negocio críticas.
Pruebas de integración	La comunicación entre controladores, servicios y repositorios fallaba cuando faltaban atributos en el modelo.	Bloqueo de vistas y errores 500.	Añadir pruebas manuales de flujo completo: login → home→ activos → movimientos.	Verificar siempre que las vistas reciban los atributos necesarios.

Tipo de Impedimento	Descripción del Problema	Impacto en el Sprint	Estrategia de Solución	Medidas Preventivas / Lecciones Aprendidas
Configuración del Dockerfile	La aplicación no arrancaba dentro del contenedor por rutas incorrectas y falta de JAR final.	Bloqueo del despliegue en Docker.	Crear Dockerfile desde base openjdk:25-slim, copiar el JAR y definir ENTRYPOINT.	Validar el JAR antes de construir la imagen. Mantener Dockerfile simple y documentado.
Conexión Spring Boot - MySQL en Docker	Spring Boot no lograba conectarse al contenedor MySQL por usar localhost en lugar del nombre del servicio.	La aplicación no iniciaba.	Cambiar la URL JDBC a: jdbc:mysql://mysql:3306/zeertydb.	Usar siempre nombres de servicio de Docker Compose en lugar de IPs o localhost.
Persistencia de datos	La base de datos se borraba al reiniciar contenedores.	Pérdida de datos del MVP.	Añadir volumen persistente: mysql_data:/var/lib/mysql.	Documentar el uso de volúmenes. Nunca usar contenedores sin persistencia en producción.
Orden de arranque de contenedores	Spring Boot arrancaba antes que MySQL y fallaba la conexión.	Reinicios constantes del contenedor.	Añadir depends_on: - mysql en Docker Compose.	Validar orden de arranque en entornos dockerizados.

Tipo de Impedimento	Descripción del Problema	Impacto en el Sprint	Estrategia de Solución	Medidas Preventivas / Lecciones Aprendidas
Pruebas en entorno Docker	Las pruebas manuales no coincidían con el comportamiento en local (H2) y en Docker (MySQL).	Resultados inconsistentes.	Crear dos modos de ejecución: H2 (desarrollo) y MySQL (Docker). Probar ambos.	Mantener entornos separados: desarrollo, pruebas y despliegue.
Datos inconsistentes entre entornos	La base de datos H2 generaba datos distintos a MySQL.	Fallos en auditoría y movimientos.	Unificar esquema y validar con Flyway/Liquibase.	Documentar diferencias entre H2 y MySQL.
Pruebas de rendimiento en Docker	El arranque inicial era más lento en contenedores.	Percepción de lentitud del MVP.	Ajustar configuración de MySQL y revisar logs.	Considerar tiempos de arranque en entornos dockerizados.

Conclusiones del Sprint 4

El Sprint 4 finalizó con un incremento funcional estable, cumpliendo los objetivos planteados y consolidando la base técnica del MVP. Durante el desarrollo se completaron las Historias de Usuario previstas, se resolvieron los impedimentos técnicos y se validó el funcionamiento tanto en entorno local como en entorno Dockerizado.

A nivel técnico, se logró una arquitectura consistente, con separación clara entre capas (controladores, servicios y repositorios), integración correcta de Spring Security y persistencia mediante H2/MySQL según el entorno. La dockerización del proyecto permitió disponer de un entorno reproducible, aislado y con persistencia real mediante volúmenes, lo que facilita el despliegue y las pruebas en máquinas externas.

Las pruebas funcionales y de integración confirmaron que los flujos principales del MVP (login, gestión de activos, registro de movimientos y auditoría básica) funcionan de forma coherente y estable. Se solventaron problemas relacionados con rutas, vistas Thymeleaf, atributos del modelo y conexión con la base de datos, reforzando la calidad del incremento entregado.

En cuanto a la gestión del Sprint, se mantuvo una buena cadencia de trabajo, con comunicación clara de impedimentos y resolución ágil.
La documentación generada (manual técnico, arquitectura, glosarios y README) deja el proyecto preparado para futuras iteraciones.

En resumen, el Sprint permitió:

- Consolidar la base funcional del MVP.
- Completar la dockerización y asegurar persistencia.
- Validar el sistema mediante pruebas funcionales y de integración.
- Resolver impedimentos técnicos clave.
- Documentar correctamente el incremento.

Con todo esto, el proyecto queda preparado para realizar el video final del MVP.

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.
- Documentación oficial de Spring Boot Security: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#web.security
- https://github.com/sergarb1/CursoIntroduccionADocker
- https://martinfowler.com/articles/practical-test-pyramid.html
- https://docs.spring.io/spring-framework/reference/testing/mockmvc.html
- https://site.mockito.org/
- https://docs.spring.io/spring-boot/reference/testing/index.html#testing
- https://docs.junit.org/6.1.3/overview.html